Less sitting idle.
More possibilities.

ResourceShare connects students, resource owners and administrators through a complete, traceable borrowing workflow for underused campus equipment and spaces.

STAGING RELEASE / 1 NOVEMBER 2026 (INDIA) / REVIEW BEFORE SUBMISSION

Problem statement

Unlock Underused Equipment and Spaces. A trusted platform for institutions and communities to share resources safely and fairly.

Verified deliverables

Shared D1 database, protected lifecycle operations, conflict prevention, FIFO promotion, condition reports, owner-verified returns, audit verification and a public isolated judge demo. Eighteen automated tests and a live HTTP workflow test pass.

Read this disclosure first

This is an upgrade to pre-existing ResourceShare code. All 50 requested features are accounted for: 20 implemented and tested, 22 partial, 3 demonstration-only and 5 planned. Google OAuth, email delivery, runtime AI and scheduled background jobs are not configured. Official build-window dates and team details still need confirmation.

Live staging: https://resourceshare-upgrade-staging.electhill.chatgpt.site
GitHub: roshnihia-cell/ResourceShare-Hackathon
Branch: resourceshare-50-features-upgrade

01 / Executive summary and selected use case

ResourceShare starts with a single campus: students borrow physics laboratory space, projectors, microscopes, cameras, computing equipment and a seminar hall. Owners retain approval authority; administrators oversee identity, qualifications, policy and disputes. The full request-to-return state machine persists in a shared database. Judges can operate every role in a clearly labeled isolated fictional workspace.

02 / Why the problem matters

Valuable resources can remain idle because availability, permission and responsibility are unclear. Informal messaging produces conflicting requests, invisible queues and little evidence of equipment condition. ResourceShare makes access reviewable and records responsibility at both ends of a borrowing.

03 / Target users and adoption

Student borrowers need discoverability, transparent access and clear pickup/return expectations. Resource owners need qualification checks, scheduling control and condition evidence. Campus administrators need policy, incident oversight and utilization signals. Community institutions could adopt the workflow after identity, data retention and operational responsibilities are formalized.

04 / Complete borrowing lifecycle

01 / Owner lists a resource

Category, location, safety, qualification and deposit record.

02 / Student requests a time

Eligibility, purpose, agreement, request cap and interval checks.

03 / Owner reviews the request

Approve or reject with a reason. Conflicts join FIFO.

04 / Borrower checks in

Private time-limited pass plus before-use condition.

05 / Borrower checks out

After-use condition moves the booking to return pending.

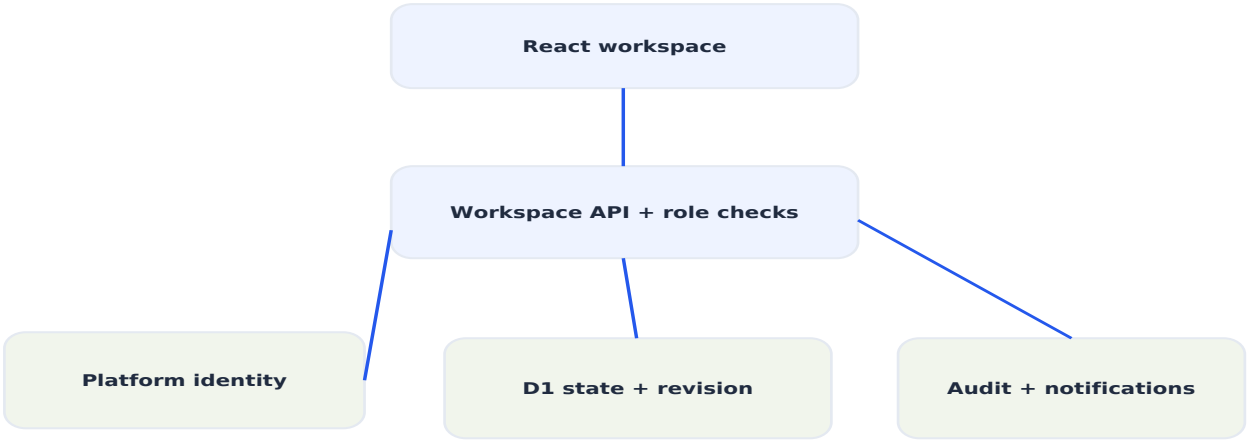
06 / Owner verifies return

Inspection note completes the handover and promotes a compatible queue request.

07 / History stays reviewable

User-scoped notifications, receipt and hash-linked event history.

05 / System architecture



The static React/Vite prototype is preserved at /legacy. The new server-capable React/Vinext starter adds a Cloudflare Worker API, D1 database and trusted hosting identity boundary. A supported server runtime was necessary because browser-only state cannot enforce authorization or prevent competing allocations.

06 / Database structure

One physical workspaces table stores id, revision, data and updated_at. The JSON data document contains profiles, resources, bookings, incidents, training evidence references, feedback, notifications, policy and audit events. campus:one identifies the real authenticated workspace; a random demo cookie identifies each isolated fictional workspace.

Each mutation loads the current revision, authorizes the actor, applies domain checks and writes a new document only if the revision is unchanged. Zero affected rows returns HTTP 409. Booking and audit updates therefore share one atomic state update. This intentionally small design serializes writes and requires normalization/pagination for larger deployments.

07 / Authentication and authorization

The platform supplies stable user ID and email headers after ChatGPT sign-in. New real users start as unverified students. The authenticated Site owner is bootstrapped using a server-only admin allowlist; administrators review membership and can grant owner roles. Request bodies cannot set real roles. Every mutation checks ownership or role again on the server. Demo role selection applies only to its isolated workspace.

Google/Gmail OAuth and application email verification remain unconfigured. They are not represented as working integrations. Production HTTPS demo cookies are HttpOnly, Secure and SameSite=Strict; local HTTP preview has a named-host exception.

08 / Fairness and allocation

FIFO uses a monotonic request sequence. Overlapping live requests enter the waiting list. An owner cannot approve over an allocated interval or bypass an earlier overlapping pending/waiting request. Cancellation, verified return, maintenance completion or an operational no-show sweep promotes the earliest eligible compatible request. Promotion produces a new request for human approval, not automatic permission to enter.

Administrators configure maximum open requests per person, maximum duration and no-show grace. Jain's index is (sum of allocation counts)^2 divided by n times the sum of squared counts. Empty/all-zero counts display 1 by convention. This measures allocation-count distribution, not differences in need, duration or entitlement.

09 / Safety, privacy and human oversight

Resource-specific instructions and an exact agreement snapshot are saved with each request. Before-use and after-use reports are separate from the owner's return verification. Training evidence references require administrative review. Incidents retain original descriptions and require human resolution notes. Urgent reports reach the in-app inbox only; immediate danger must be handled through campus security directly.

Student responses omit other people's booking details, profile records and private passes. The demonstration data is fictional. Deposits are ledger entries with collection/refund guards; no money is collected. No biometrics, real student identity documents or external emergency contacts were introduced.

10 / Audit integrity and operational limits

Events contain previous and current SHA-256 hashes. Administrator verification detects inconsistent changes within the saved chain. A privileged database operator could regenerate the chain, so this is not an independently anchored or immutable audit service. Database authorization, access review and retention remain essential.

The campus state document and full event history grow with activity. The application imposes a 10,000-event guard, but production needs rate limiting, demo cleanup, normalization, indexes, pagination, backups and retention controls. New institution federation and performance under real campus load are not verified.

11 / AI tools and external services

ChatGPT/Codex assisted implementation, design, tests, documentation and deployment. There are no runtime model calls. Kit suggestions are fixed transparent rules and incident text is human-authored. The technology stack includes React, Vinext/Vite, Cloudflare Workers/D1, Drizzle, lucide-react and qrcode-generator; the dependency lockfile records exact versions. Google Fonts has system-font fallbacks.

12 / Testing evidence

The original four booking tests and original Vite build passed before development. The upgraded suite now has 18 passing tests, covering successful lifecycle, queue promotion, role/ownership rejection, invalid intervals and caps, pass validation, qualification review, incident resolution, deposit/refund and feedback, immutable agreement snapshots, admin role assignment, audit edits, SQLite stale-writer behavior, no-show release and extension conflicts.

The live HTTP integration test passed against the public staging origin. It exercised actual D1 persistence, owner resource listing, student authorization rejection, stale revision 409, FIFO waiting, authenticated-demo check-in/out, owner return verification, compatible queue promotion, full audit verification and demo exit. A separate browser test exercised request, approval, condition reporting and owner-confirmed completion on local preview.

A preview-only Secure-cookie issue was discovered in browser testing and repaired for named local HTTP hosts. Production HTTPS retains the secure-cookie flags. The browser also confirmed resource search. Interactive ChatGPT sign-in/logout, physical-device responsiveness, physical-camera scanning and formal accessibility compliance have not been certified.

13 / Judge access and demonstration

Open the staging site and choose Try the judge demo. Asha and Kavin are trained fictional students, Maya is deliberately untrained, Dr. Meera is the resource owner and Campus Admin is the reviewer. The dropdown switches only demo identities. Request an interval that includes the planned check-in time; future bookings cannot check in early. Use the judge guide for a complete four-minute workflow and negative checks.

14 / Known limitations

Google OAuth, email delivery, runtime AI recommendations/summaries, recurring reminders and cross-institution federation are planned. File uploads, smart calendar, maintenance logs, payment processing, multi-party authorization, outbound owner handover acknowledgment, translation and offline synchronization are partial. QR codes produce access deep links; scanning in another browser does not transfer the session. The map and kit suggestions are demonstration illustrations. The 50-feature appendix gives exact statuses.

15 / Improvements and adoption potential

Before a campus pilot: connect official membership, configure supported identity/email, add private evidence storage and trusted schedules, formalize security contacts and return procedures, define retention/backup policies and independently review authorization/accessibility. Collect representative activity before evaluating forecasts. Use human review for any AI-assisted incident summaries or consequential access decisions.

16 / Originality and build-window disclosure

The source baseline is b689b07a852aac10056181bc30fbe5adccc39370 on resourceshare-final-submission. It already contained booking, queue, fairness, audit and UI code developed with earlier ChatGPT assistance. Both existing branches have unrelated history but identical application source. The new resourceshare-50-features-upgrade branch preserves its actual baseline parent and commit history; no timestamps or earlier work are fabricated.

The supplied Sites React/Vinext starter and open-source libraries are disclosed. The official 48-hour dates and timezone have not been supplied. The team must identify components developed before, during and after the official window, and verify compliance with organizers before submitting. This report makes no all-original-in-48-hours claim.

17 / Team information awaiting confirmation

Team name: to be supplied

Team leader and full member names: to be supplied

College/institution: to be confirmed

Submission contact email: to be supplied

Actual role and contribution of every member: to be supplied

Official build-window start/end and timezone: to be supplied

18 / Deployment and reproducibility

The new Site has its own staging identity and D1 database. The earlier live site is not replaced. Authored source is committed to the dedicated GitHub branch and the corresponding Site source repository; the hosted archive is built from the Site repository commit. Node 22.13+ and pnpm are required. Apply the generated D1 schema before local preview. The old static Netlify/Vercel settings do not deploy this Worker branch.

19 / Submission materials

This report accompanies the judge guide, complete feature report, architecture diagram/schema description, test results, demo recording script, environment template, deployment/troubleshooting instructions, AI/prior-code disclosure and draft submission answers. No hosted video has been produced. The provided Google Form could not be retrieved through the search tool; the answer draft is adaptable and the form has not been submitted.

20 / Review before final submission

Confirm team details and the official build window. Review the truthful feature status and evidence, record/upload a demonstration if required, and inspect the final Google Form before submitting. Existing branches remain unmerged and the previous site remains intact.

Appendix / All 50 requested features

Implemented and tested describes implemented behavior with automated or recorded workflow coverage. Partial and demonstration-only entries must not be presented as full production features.

#	Feature / status	Evidence or limitation
1	Google/Gmail sign-in Planned	Not configured. Supported ChatGPT identity is available instead; Gmail OAuth was not tested.
2	Email verification Partial	Platform identity supplies an email. No application email verification flow or email delivery provider.
3	Student and owner profiles Implemented and tested	Server profiles, editing, student verification and owner identity in isolated demo; production owner assignment requires admin.
4	Role-based dashboards Implemented and tested	Server authorization plus student, owner and administrator views. Demo role switching is sandbox-only.
5	Resource listing and management Partial	Owner listing and maintenance available. Original add/remove prototype preserved at /legacy; new resource editing/deletion not implemented.
6	Search and filters Implemented and tested	Resource directory, category filter and text search; browser verification recorded separately.
7	Shared database availability Implemented and tested	D1 workspace state and atomic revision checks. Independent demo sandboxes do not share reservations.
8	Date/time booking requests Implemented and tested	Validated start/end, future-time checks, purpose and agreement acceptance.
9	Owner review Implemented and tested	Owner approval, rejection reason, notification and state transition guards.
10	Eligibility verification Implemented and tested	Institutional status and approved training checked server-side.
11	Fair allocation Implemented and tested	FIFO review, open-request cap, duration cap and explainable distribution metric.
12	FIFO waiting list Implemented and tested	Conflicts queue automatically; compatible requests promote for owner approval after cancellation or verified return.
13	Email notifications Planned	In-app notifications work. No email provider, deliveries or email tests.
14	QR check-in Partial	QR generation and deep link available; borrower, token and time are checked on check-in. Physical camera scanning not tested.
15	QR check-out Partial	Same deep link opens checkout for a checked-in borrower. Physical camera scanning not tested.
16	Before-use reports Implemented and tested	Required borrower condition text captured at check-in.
17	After-use reports Implemented and tested	Required checkout report followed by independent owner verification.

#	Feature / status	Evidence or limitation
18	Damage/incident reporting Implemented and tested	Booking-scoped reports, severity and in-app reviewer routing.
19	Cancellation and extensions Implemented and tested	Cancellation promotes queue; extension approval rejects conflicts.
20	Tamper-resistant audit Partial	Server-authorized hash-linked events, integrity verification and export. Database operators can rewrite the chain; no external anchor.
21	Admin utilization analytics Partial	Request, allocation, return counts and distribution. Time-based utilization percentages not implemented.
22	Digital student ID Demonstration-only	Profile card with review status; no connection to an official campus identity system.
23	Smart calendar Partial	Shared resource schedule with local dates and time selection; no month-grid drag/drop or calendar integration.
24	Maintenance tracking Partial	Owner maintenance status blocks new bookings; no service logs or recurring maintenance plans.
25	AI recommendations Planned	Transparent rule-based project-kit suggestions only; no AI API.
26	Institutional email-domain verification Partial	Manual institutional membership review implemented. Automatic domain verification not implemented.
27	Refundable deposits Partial	Collection/refund bookkeeping with lifecycle guards. No payment processing.
28	Safety instructions/tutorials Partial	Per-resource safety text and accepted agreement; video tutorials not added.
29	Emergency escalation Partial	Urgent in-app incident routing. No external security dispatch, SMS, telephone or guaranteed response.
30	Multi-institution sharing Planned	Institution labels and workspace partitioning exist; federation and cross-institution permissions not implemented.
31	Ratings/feedback Implemented and tested	Completed-booking-only ratings, duplicate prevention and resource feedback display. Extended lifecycle automated test included.
32	Automated overdue reminders Partial	On-demand operational sweep creates in-app reminders. No scheduler or background email service.
33	Digital agreements Implemented and tested	Exact accepted agreement and timestamp saved on each booking.
34	PDF receipts Partial	Client-generated PDF download available; report rendering and receipt download checks documented separately.
35	Accessible responsive UI Partial	Semantic controls, labels, focus trap, keyboard dismissal, responsive CSS and two themes. No full accessibility certification or physical-device audit.
36	Dispute resolution Implemented and tested	Administrator incident resolution with required review note and audit event.

#	Feature / status	Evidence or limitation
37	Conflict detection Implemented and tested	Overlap checks, owner approval guards and atomic compare-and-swap prevent stale writes.
38	Campus map Demonstration-only	Illustrative fictional campus positions, clearly labeled; no geolocation/navigation.
39	Project bundles Demonstration-only	Suggested project kits only. Each item requires an independent reservation.
40	Group bookings Partial	Team/participant text attached to one responsible borrower. No co-borrower membership or group authorization.
41	Training certificate uploads Partial	Evidence references and admin verification available. Binary file uploads not implemented.
42	Time-limited passes Implemented and tested	Private random token tied to borrower, approved booking and permitted check-in interval.
43	Handover confirmation Partial	Borrower check-in and owner-verified return work. Separate owner acknowledgment of outbound handover not implemented.
44	Automatic no-show release Partial	Server on-demand sweep releases no-shows and promotes queue. No unattended scheduler.
45	Configurable fair-use limits Implemented and tested	Admin policy controls request cap, duration and no-show grace.
46	AI incident summaries Planned	Original incident text is shown to human reviewers. No model-generated summary.
47	English/Tamil interface Partial	Primary navigation and greeting translated; forms and supporting content remain English.
48	Offline condition drafts Partial	Device-local draft text survives reopening. Manual submission after reconnection; no service-worker cache or automatic sync.
49	Booking/safety policies Implemented and tested	Admin borrowing policy, resource safety instructions and immutable accepted agreement snapshots.
50	Demand forecasting Partial	Observed demand counts and queue pressure displayed. Predictive forecasting not implemented.